Agent configuration, prompt skills, and MCP tooling for AI-assisted development

The most impactful investment is not building more agents — it is engineering their context. Across all three research areas, the dominant finding is that context quality determines agent effectiveness far more than prompt sophistication. Frontier LLMs can follow roughly 150–200 instructions before quality degrades uniformly, and Claude Code's system prompt already consumes ~50 of those slots. humanlayer Every line in a system prompt, every MCP tool definition, and every continuation document competes for a finite cognitive budget. The research yields a clear priority order: configure the continuation prompt skill first (it compounds across all sessions), then the pre-commit and security diagnosis agents (they prevent the highest-cost defects), then the remaining agents and skills incrementally. MCP tool definitions alone can consume **72% of a 200K context window** if loaded indiscriminately — making dynamic toolset loading and the code-execution pattern essential, not optional.

The evidence base is uneven. Security review agents and MCP integration patterns are well-documented by production teams. Architecture review and performance diagnosis agents have emerging tooling but limited production benchmarks. TDD review agents at the planning gate represent the largest evidence gap — no production-grade standalone tool exists. Where consensus is absent, this report states so explicitly and provides the strongest available evidence.

Research Area 1: Specialist review agents

Architecture review works best on full plans with structured output

Top recommendation: Invoke the Architecture Review Agent at the planning gate on the complete implementation plan (not per-ticket), requiring as input the ERD, module dependency map, and any existing ADRs. Structure output as severity-bucketed findings (Critical/High/Medium/Low) with specific component references and actionable remediation, not narrative commentary.

The strongest production implementation is the **Microsoft Architecture Review Agent** (open-sourced February 2026), which ingests architectural descriptions in any format and produces prioritized risk analysis with component dependency mapping including fan-in/fan-out metrics and orphaned component detection. Microsoft Community Hub ArchGuard's Co-mate project from the Thoughtworks open-source community takes the complementary approach — maintaining a persistent architectural model and checking changes incrementally against it. La Times A third pattern, the ArchGuard CLI, validates code changes against ADRs using semantic similarity plus LLM reasoning, outputting specific violations with line references. GitHub

The **full-plan vs. per-ticket trade-off** is real but resolvable. Full-plan review catches systemic issues (single points of failure, coupling anti-patterns, missing components) that per-ticket review misses entirely. Microsoft Community Hub Per-ticket review is cheaper and faster but blind to

cross-cutting concerns. The recommended pattern is full-plan review at the planning gate, with lightweight per-ticket validation during implementation using ADR compliance checks.

System prompt structure (targeting 150–250 words): role definition → context specification (what artifacts are provided) → evaluation criteria (SOLID principles, coupling metrics, schema separation) → structured output format → scope boundaries → 2–3 few-shot examples of good findings. GitHub Security Lab's taskflow architecture explicitly recommends **task-specific prompts over one mega-prompt**, (GitHub) and this principle applies to architecture review.

Anti-pattern: Generic, ungrounded commentary. When the agent lacks module structure or dependency information, it produces boilerplate advice ("consider using dependency injection") rather than actionable findings. The ArchGuard CLI mitigates this with a `--ci` flag that warns on truncated input rather than hallucinating analysis. (GitHub)

Security review requires deterministic SAST as input, not replacement

Top recommendation: Structure the Security Review Agent as a **triage layer over deterministic static analysis**, not as a standalone reviewer. Feed it Bandit/pip-audit/Semgrep output alongside the endpoint plan. Map OWASP Top 10 categories directly into the system prompt as evaluation criteria. Invoke on every PR touching auth, payments, or external APIs.

The evidence here is the strongest of any agent category. **Semgrep Assistant** achieves **96% agreement with human security researchers** on true positive identification (Semgrep) by combining deterministic SAST with LLM reasoning. (Semgrep) By contrast, pure LLM security review produced **88% false positives** when applied to IDOR detection without SAST grounding (Semgrep's own benchmark). (Semgrep) The hybrid pattern — deterministic tools find candidates, LLM triages and explains — is now industry consensus.

The **minimum viable context set** (derived from GitHub Security Lab's taskflow architecture) consists of five elements: component inventory with entry points and trust boundaries, the changed code/diff with surrounding context, data flow information (sources to sinks), the dependency manifest, and previous triage decisions. Semgrep's "Assistant Memories" feature — storing previous triage decisions for reuse — is a pattern worth replicating. (Semgrep)

For **OWASP mapping**, the system prompt should enumerate the categories the agent can reliably detect. LLMs are strongest on injection (A03), broken access control (A01 in simple cases), and cryptographic failures (A02). They struggle with security misconfiguration (A05) and SSRF (A10), which require architectural context. (Markaicode) The OWASP LLM Top 10 applies when the system being reviewed includes LLM components — tools like Promptfoo implement these as configurable test presets. (promptfoo)

Anti-pattern: Running LLM security review in CI/CD pipelines without input sanitization. Aikido Security's "PromptPwnd" research found that untrusted user input injected into LLM prompts in GitHub Actions can lead to credential theft, affecting at least 5 Fortune 500 companies. (Aikido) The review agent itself becomes an attack surface.

TDD review has the weakest tooling but the clearest evaluation criteria

Top recommendation: Build the TDD Review Agent to validate acceptance criteria against a testability checklist derived from BABOK quality characteristics, operating on story files with optional conftest context. The agent should flag ambiguity patterns rather than generating test code.

This is the **least mature agent category**. No production-grade, widely-adopted tool exists specifically for pre-implementation test plan review. The closest implementations are a Requirements Verification Megaprompt pattern, the Meridian RFC (a GitLab agent validating merge requests against acceptance criteria), and academic work from ICEIS 2025 on acceptance criteria validation using NLP.

The **testability checklist** is well-established even without dedicated tooling. Testable criteria use Given/When/Then structure with concrete preconditions and measurable outcomes. Substack Parallelhq Common ambiguity patterns that the agent should flag include: vague qualifiers ("make it fast"), missing error scenarios, no measurable outcomes ("user is satisfied"), implementation-coupled criteria ("use Redis"), and missing preconditions. The TDAD paper (2026) found that a vanilla coding agent caused **562 pass-to-pass test failures across 100 instances** arXiv — validating the test plan before implementation is high-leverage.

Story files alone suffice for syntax validation, completeness checking, and ambiguity detection. Architecture/conftest context becomes necessary for validating that test scenarios match actual API contracts, checking fixture consistency, and detecting missing integration test coverage.

Anti-pattern: Generating test code during the review phase. The TDD review agent should produce a validation report on the test plan, not test implementations. Mixing review and generation leads to the agent "fixing" ambiguous criteria by making assumptions rather than flagging them.

The pre-commit agent must explain errors through structured output, never auto-fix

Top recommendation: Implement the Pre-commit Check Agent using Claude Code's custom mode or hook system with file-editing tools explicitly excluded. Output format should be structured as `ERROR → ROOT CAUSE → OPTIONS (with trade-offs)`, not conversational. Use PreToolUse hooks to block Edit/Write tools entirely.

The **explain-don't-fix pattern** is implemented in Cursor AI's "Ask Mode" (read-only, no code changes), AltexSoft JetBrains AI Assistant's "Explain with AI" feature, JetBrains and Claude Code's custom modes with tool restrictions. The key architectural insight is **mode separation**: explanation and modification must be physically separated by restricting available tools, not by instruction alone. System prompt instructions to "never modify files" are insufficient — models exhibit sycophantic drift and start suggesting fixes despite instructions. Substack

For **CLI output format**, structured is preferred over conversational. The recommended structure per error:

- **Error**: type + file:line reference

- **Root cause**: concise explanation referencing the relevant rule
- **Options**: 2–3 resolution approaches with trade-offs for each
- **Priority**: errors ordered by severity (blocking → warning → info)

Claude Code hooks provide the enforcement mechanism: (GitHub) PostToolUse hooks on Edit/Write/MultiEdit for running quality checks after edits, (Eesel AI) with exit code 2 to block further actions. The blog.sshh.io practitioner recommendation is critical: **block at submit time, not at write time** — let the agent finish its plan, then check the final result. (Sshh)

Anti-pattern: Running formatters in PostToolUse hooks that consume context window tokens on every edit. Style enforcement should use deterministic tools (ruff, black) via pre-commit hooks outside the agent's context, not through the agent itself. (Humanlayer +2)

Hallucinated package detection is the highest-urgency security concern

Top recommendation: Integrate Socket.dev's MCP server (`https://mcp.socket.dev/`) for real-time dependency scoring, combined with a verification step that checks package existence, download count, creation date, and maintainer history. Flag any package with fewer than 1,000 downloads, less than 6 months of history, or Levenshtein distance ⩽ 2 from a known package name.

The USENIX Security 2025 paper documents a **19.7% overall hallucination rate** across 2.23 million package recommendations from 16 LLMs — **440,445 hallucinated packages** including 205,474 unique non-existent names. Commercial models hallucinate at 5.2%, open-source at 21.7%. (USENIX) Critically, **43% of hallucinated packages appeared consistently** across repeated prompts, making them predictable targets for slopsquatting attacks. (Andrew Nesbitt)

The weaponization is already real. A hallucinated `huggingface-cli` package uploaded to PyPI received **30,000+ downloads in 3 months** after Alibaba copy-pasted the hallucinated install command into a public repository. (Aikido) The term "slopsquatting" (coined by Python PSF Developer-in-Residence Seth Larson) now describes the attack pattern. (Security Boulevard) Andrew Nesbitt's December 2025 research shows slopsquatting combining with dependency confusion attacks, with 404 logs from package registries serving as "toxic waste" revealing which names attackers should squat. (Andrew Nesbitt)

Tooling responses as of early 2026:

- **Socket.dev MCP** — hosted at `https://mcp.socket.dev/`, no API key required, supports npm and PyPI, provides supply chain risk scores
- **SlopGuard** (open-source) — 8 anomaly detection algorithms including namespace squatting and typosquatting detection, 70% fewer API calls via lazy loading (Aditya01933)
- **Trend Micro research** — found that even MCP-augmented vibe-coding reduces hallucination rate by ~50% but doesn't eliminate it

Anti-pattern: Relying solely on package existence checks. Malicious actors pre-register hallucinated names (the core slopsquatting attack), so existence alone doesn't mean safety. (Trend Micro) The agent must also check download count, age, maintainer history, and source repository linkage.

Performance diagnosis maps load test output to SLO burn rates, not raw thresholds

Top recommendation: Structure the Performance Diagnosis Agent around Google SRE's multi-window burn rate alerting pattern rather than simple threshold checks. Input should include p50/p95/p99 histograms, error rates, traffic volume, saturation metrics, and deployment markers with commit hashes. Output should specify burn rate, time to error budget exhaustion, suspected causes with evidence, and recommended remediation.

Datadog's Bits AI SRE is the most comprehensive production implementation — it forms hypotheses, gathers telemetry iteratively, and correlates signals across logs, metrics, traces, and source code. (Dash0) Its **"Suspect Commits" feature** directly maps performance regressions to specific code changes using deployment tracking. The pattern is: tag every deployment with version/commit hash → compare golden signals between consecutive versions → identify the commit window where regression appeared.

The **standard input/output contract** for p99-latency diagnostics follows this structure: SLO definition (service, SLI metric, target, window, error budget) (SigNoz) → current metrics (percentile histograms, error rate, traffic, saturation) → recent deploys with commit references → triage logic (check burn rate, correlate with deployments, examine dependencies, check saturation) → structured output (severity, burn rate multiplier, budget exhaustion ETA, suspected causes ranked by confidence, recommended actions).

k6 provides direct CI integration through its thresholds system — non-zero exit codes on threshold breach that can block PR merges. (Frugal Testing) Grafana Cloud k6 now offers AI-powered insights connecting test outcomes to SLOs. **Locust** reports per-endpoint p50/p90/p95/p99 with CSV export for downstream analysis.

Anti-pattern: Averaging percentiles across time windows. This is mathematically incorrect and produces misleading results. Use raw histograms (OpenTelemetry bucket-based) for percentile computation. Google SRE explicitly warns against percentile aggregation.

Specialist agents outperform generalists when scoped to fewer than 200 instructions

The empirical evidence on specialization vs. generalism converges on a clear finding: **specialist agents with focused, short system prompts outperform general-purpose agents with large context dumps, but only when the task boundaries are well-defined.**

The AgentCoder ablation study (ACL) found that separating programmer and test designer roles into distinct agents improved state-of-the-art performance. MapCoder (ACL 2024) confirmed that "every agent has its role in the pipeline as turning off any agent reduces performance." (ACL Anthology) MALMM (IROS 2025) found that "a single agent performing multiple roles struggles to excel in all of them" and proposed role-specific contexts.

However, Anthropic's own guidance from "Building Effective Agents" (December 2024) cautions: **"The most successful implementations weren't using complex frameworks. They were building with simple, composable patterns."** (Anthropic) The Agentless approach (simple localization-repair without multi-agent complexity) demonstrated competitive performance on SWE-bench, (arXiv) and Anthropic explicitly recommends starting with the simplest solution. (Medium)

The **optimal system prompt length** is constrained by two findings. First, HumanLayer's analysis (citing arXiv 2507.11538) shows frontier thinking LLMs can follow **~150–200 instructions with reasonable consistency**, with quality degrading uniformly as count increases. (Humanlayer) (humanlayer) Second, the "Same Task, More Tokens" paper (ACL 2024) found reasoning capabilities degrade even at **3,000 tokens of input**, far below technical maximums. (arXiv) (arXiv) The practical recommendation: specialist agent system prompts should be **100–300 words** (roughly 150–400 tokens), with the total instruction budget across CLAUDE.md, system prompt, and MCP tool descriptions staying well under 200 instructions.

Research Area 2: Reusable prompt skills

Socratic PRD elicitation requires enforced ask-before-generate boundaries

Top recommendation: Structure the PRD skill system prompt using the Flipped Interaction Pattern: the agent asks questions one at a time before generating any content. Use breadth-first sequencing (sweep across user roles, then edge cases, then technical constraints) before depth-first drilling into any single area. Place the questioning behavior in the system prompt; provide the output template in the user turn.

The most reliable enforcement mechanism is explicit behavioral gating: `"You MUST ask at least 8 clarifying questions across all user roles and edge cases before generating any specification content. If you generate content before questions are resolved, you have failed your task."` Moxo's production agent prompts use: `"Before executing any action, verify all required inputs. If uncertain, ask for clarification rather than guessing."` (Moxo) The Cognitive Verifier Pattern (documented by Zapier) tells the AI to break questions into smaller parts and get feedback before proceeding. (Zapier)

Question sequencing should follow the Illumination → Confrontation → Clarification → Integration cycle from the AI-Augmented Socratic Method. (Emergent Journal) Start with open questions about user roles and intended workflows (breadth), then challenge assumptions with "what happens if..." scenarios (depth), then clarify ambiguities, then synthesize. Critically, **ask 2–3 questions at a time** — asking all questions at once overwhelms the developer, while one-at-a-time is too slow.

ChatPRD is the most mature production tool implementing this pattern, (Substack) with MCP integration for Claude Code and direct conversion to Linear/Jira tickets. Fireside PM's 2025 benchmark ranked Claude #1 among 5 AI tools for PRD quality, citing "strongest strategic thinking." (Substack)

Anti-pattern: Leading questions that presuppose architecture. "Should we use REST or GraphQL?" constrains thinking; "What are the integration requirements?" opens exploration. Also avoid letting the agent generate speculative content mid-interview — models strongly tend toward eager generation.

Ticket generation should use tool calling, not JSON mode, with two-step reasoning

Top recommendation: Use API-level structured outputs via tool/function calling (not raw JSON mode) for ticket generation. Implement a two-step approach: let the model reason freely about the PRD first, then convert reasoning into structured ticket format. Validate output against Pydantic schemas before import.

Critical research finding from Tam et al. (EMNLP 2024 Industry Track): **JSON-mode constrained decoding degrades reasoning by 10–15%** because models are forced to output answers before completing chain-of-thought. The fix is to always place reasoning fields before answer fields in schemas, or better, separate the reasoning and structuring steps entirely.

The Instructor library benchmarks (2024) found that **Claude models see significantly greater improvement with few-shot examples** compared to GPT-4o variants for structured output. (Instructor) For batch ticket generation specifically, few-shot examples of 2–3 complete tickets in the target format consistently outperform detailed format instructions alone.

Each ticket should contain: epic reference, title, type (story/task/bug), description, acceptance criteria in Gherkin format, API endpoint specifications (path, method, query/path/body parameter labels), response schema reference, estimated complexity, and dependency references. The PRD-to-ticket pipeline documented by Product Direction adds a **refinement agenda** — categorizing items as ALREADY DECIDED (from PRD), VALIDATE WITH TEAM (technical decisions), or OPEN QUESTIONS (need input) — which prevents the agent from making assumptions about unresolved issues. (Substack)

No native bulk creation exists in Linear MCP. The `create_issue` tool processes one issue at a time. For a PRD generating 15–30 tickets, the agent must call `create_issue` iteratively. This is a meaningful constraint — plan for sequential creation with parent-child relationships established via `parentId`.

Anti-pattern: Forcing JSON structure during the reasoning phase. Let the model analyze the PRD and identify ticket boundaries in natural language first, then format into the target schema. Also avoid generating tickets from partial PRDs — incomplete input produces speculative acceptance criteria.

Continuation prompts should follow the state-snapshot-plus-pointer pattern

Top recommendation: Generate handoff documents with a fixed structure: context (1–2 sentence goal), progress (completed items with outcomes), current state (blockers, key decisions), "do not redo" list, next steps (prioritized with specific file/function references), environment commands, and constraints. Store in `.claude/handoffs/` with timestamps. (LobeHub) Compact at 60% context, not 90%. (SitePoint)

Anthropic's engineering team provides the authoritative guidance: **"The core challenge of long-running agents is that they must work in discrete sessions, and each new session begins with no memory of what came before."** (Anthropic) Their recommended architecture uses an initializer agent plus a coding agent that "makes incremental progress in every session, while leaving clear artifacts for the next session." (Anthropic)

The **Document and Clear pattern** (widely adopted in the Claude Code community) is: have Claude dump its plan and progress into a `.md` file, `/clear` the context, then start a new session pointing at that file. Trail of Bits' claude-code-config repository formalizes this: "commit your work, write a brief plan to a file, `/clear`, and start the next session by pointing Claude at that file. A fresh session with a written handoff is usually better than resuming a stale one — the context is cleaner and the prompt cache is warm." (GitHub)

Blake Link's Session Handoff Protocol provides the most detailed documented implementation: context-aware titles with measurable goals, quantifiable progress metrics, prioritized focus areas, recent accomplishments (preventing rework), development environment details, and specific expected outcomes. (blakelink) His key insight: **"Generic handoffs are worse than no handoffs. 'Continue working on the API' is less helpful than '30/90 tests passing, focus on webhook retry mechanisms in test_advanced_features_and_monitoring.py.'"** (blakelink) (Blakelink)

Anti-pattern: Relying on auto-compaction (`/compact`) for continuity. Multiple practitioners warn against this — Shrivu Shankar (Abnormal AI) states: "I avoid this as much as possible. The automatic compaction is opaque, error-prone, and not well-optimized." (sshh) (Sshh) Two compactions in a session signals the task was too large. (GitHub) (GitHub) SitePoint's guide recommends compacting at **60% context, not 90%**, and limiting sessions to work expected to consume under 120K input tokens. (SitePoint)

AI-generated code review must address the 8x duplication and 2.7x security gap

Top recommendation: Structure the C.L.E.A.R. review skill around five dimensions (Correctness, Logic/License, Efficiency, Architecture, Reliability) with three AI-specific additions: hallucinated dependency verification, duplication detection against existing codebase, and assertion depth validation in generated tests. Run automated pre-checks (SonarQube, Semgrep, dependency verification) before human review to reduce the review surface.

The evidence for AI-specific review concerns is now quantitative. GitClear's 2025 study (211 million lines analyzed) found: **refactoring collapsed from 25% of changed lines in 2021 to under 10% in 2024**, copy/pasted code rose from 8.3% to 12.3%, and duplicate code blocks rose **eightfold**. (Jonas) Short-term churn (code revised within 2 weeks) reached **7.9%** in 2024 versus 5.5% in 2020. (Jonas) CodeRabbit's analysis of 470 open-source PRs found AI-generated PRs produced **10.83 issues per PR versus 6.45 for human-only**, with **2.74× more security vulnerabilities** and significantly more readability violations. (SoftwareSeni) (CodeRabbit)

Salesforce's Prizm system (January 2026) represents the most sophisticated production response. As AI-generated PRs grew to regularly exceed 20 files and 1,000 lines, traditional file-by-file review "fractured." Prizm reconstructs developer **intent** rather than reviewing flat diffs,

using progressive disclosure, token-aware chunking, and graph-based dependency analysis. (Salesforce Engineering) (salesforce)

For test quality validation, academic research (2025) found AI-generated tests are **longer with more assertions but lower cyclomatic complexity** (median 1.00 versus human 1.31). AI tests prefer linear logic without complex branching. (arXiv) The review should check for meaningful assertions (not `assertTrue(true)`), business logic coverage, and mutation testing results.

Anti-pattern: Reviewing AI-generated code with the same process as human code. The defect profile is fundamentally different — AI code has more duplication, more stale dependencies, more readability violations against local conventions, and more omitted guardrails (null checks, early returns, exception handling). (CodeRabbit) A review process that doesn't specifically check these dimensions misses the most common AI failure modes.

AGENTS.md should be the universal source under 200 lines with progressive disclosure

Top recommendation: Use AGENTS.md as the canonical instruction file (60,000+ projects, 25+ tool integrations via the Linux Foundation's Agentic AI Foundation). (Vibe Coding +2) Keep the root file under 200 lines. Use directory-specific AGENTS.md files for package/module-specific instructions. Point to detailed docs in a `docs/context/` or `agent_docs/` subdirectory rather than inlining content. (Aihero) Symlink tool-specific files (`CLAUDE.md`, `.cursorrules`) to AGENTS.md for cross-tool compatibility.

The **progressive disclosure pattern** is consistently recommended across Anthropic, HumanLayer, Builder.io, and Trail of Bits. The root file covers WHAT (tech stack, project structure), WHY (project purpose), and HOW (build/test/verify commands) — nothing else. (sshh +2) Domain-specific instructions live in nested files that the agent discovers on demand. (Aihero +2) Claude Code's skills architecture (`.claude/skills/` with SKILL.md files) implements this at the tool level (GitHub) — skill descriptions load at session start (~100 tokens each), full content loads only when invoked. (Wmedia)

HumanLayer's critical finding: Claude Code wraps CLAUDE.md content with `"IMPORTANT: this context may or may not be relevant to your tasks."` (Claude) This means **Claude actively deprioritizes instruction file content it deems irrelevant**, making overstuffed files counterproductive rather than merely wasteful. (Humanlayer) (humanlayer)

The **amend-commit pattern** for instruction file updates: modify only the specific section that changed, commit the instruction file change alongside the code change that motivated it, use descriptive commit messages. Trail of Bits' reference implementation demonstrates the approach — global CLAUDE.md covers development philosophy and hard limits, while skills and agents encode specialized expertise. (GitHub) (GitHub)

Anti-pattern: The "monolith CLAUDE.md" — stuffing every command, convention, and guideline into a single file. HumanLayer's root CLAUDE.md is under 60 lines. (sshh) (humanlayer) Builder.io warns: "Every line in your CLAUDE.md competes for attention with the actual work you're asking Claude to do." (Builder.io) Auto-generating instruction files without refinement is

equally harmful — it floods the file with content "useful for most scenarios" that dilutes signal for the current task. humanlayer DEV Community

---

Research Area 3: MCP tool integration

Linear MCP exposes 25 tools but lacks bulk creation and blocking relationships

Top recommendation: Use the official Linear MCP server at `https://mcp.linear.app/mcp` for issue lifecycle management. Plan for iterative single-issue creation (no bulk API exists) and manage blocking relationships through the underlying GraphQL API or a community server like cosmix/linear-mcp which exposes relationship support.

The official server exposes **25 tools** covering issues, projects, comments, documents, cycles, teams, and users. speakeasy It uses **OAuth 2.1 with dynamic client registration** for interactive auth and supports API key Bearer tokens for programmatic access. Linear Rate limits are **1,500 requests/hour** per user with API key auth (250,000 complexity points/hour) using a leaky bucket algorithm. Linear

Key capability gaps for the described workflow:

- **Blocking relationships** (`issueRelationCreate` for blocked-by/blocking) are available in the Linear GraphQL API but **not directly exposed** as MCP tool parameters
- **Custom fields** are not explicitly surfaced in the MCP `create_issue` tool
- **Bulk issue creation** does not exist — even the underlying GraphQL API processes issues one at a time via `issueCreate`

For the PRD-to-Issues skill generating 15–30 tickets, the agent will need to call `create_issue` sequentially, establishing parent-child relationships via `parentId` and handling labels via `labelIds` to encode metadata (path, HTTP method, parameter types). MCP Servers Structured metadata beyond native fields must be embedded in the issue description using consistent formatting.

The February 2026 update added support for initiatives, project milestones, and project updates — useful for higher-level project management integration but not directly addressing the ticket-level workflow gaps. Linear

Anti-pattern: Assuming the MCP surface equals the full API. The Linear MCP server exposes a **curated subset** of the GraphQL API. Linear Teams needing blocking relationships, custom fields, or webhook configuration must use the GraphQL API directly or a community server. The cosmix/linear-mcp community server adds relationship support and health status tracking.

Figma MCP provides rich design context but demands aggressive token management

Top recommendation: Use `get_metadata` first for large designs to get the high-level node map, then call `get_design_context` only on specific sub-nodes. GitHub Never call

`get_design_context` on full-screen layouts — token output can exceed **350,000 tokens**, far beyond any model's useful context. Budget 3 tool calls minimum per component (metadata + context + screenshot).

The official Figma MCP server exposes **12 tools** across two deployment modes (remote at `https://mcp.figma.com/mcp` and desktop at `localhost:3845`). Figma The primary tool, `get_design_context`, returns a **structured React + Tailwind code representation** of selected Figma layers including component hierarchy, layout rules, text styles, component properties, image references, and variable names. Figma The output framework is customizable via prompts for Vue, plain HTML+CSS, or iOS. figma

Rate limits are **plan-dependent and restrictive**: 200 tool calls/day on Organization/Pro plans, **600/day on Enterprise**, and only **6 calls/month on Starter**. figma Per-minute limits range from 10–20 depending on plan tier. figma Rate limits are tied to the **plan of the file being accessed**, not the user's seat — a critical subtlety.

The **design-to-spec workflow** for architecture review follows this pattern: fetch metadata for the full page → identify relevant component nodes → fetch design context for specific components → capture screenshots for visual reference GitHub → compare against API response schemas. The `get_code_connect_map` tool (Organization/Enterprise only) retrieves mappings between Figma node IDs and code components, enabling automated design-implementation comparison.

Anti-pattern: Calling `get_design_context` on an entire page or complex frame. Documented error cases show **657,311 estimated tokens** for a single call, exceeding most client limits. GitHub The official documentation explicitly recommends breaking screens into smaller parts. Figma The community Figma-Context-MCP server by GLips implements a token-efficient alternative, simplifying and translating Figma API responses to reduce context consumption.

GitHub MCP confirms workflow log reading, enabling CI failure diagnosis

Top recommendation: Use the official GitHub MCP server with toolset filtering (`--toolsets=repos,pull_requests,actions,code_security`) to load only needed capabilities. The server **confirms workflow log reading** via `get_job_logs` with a `failed_only` parameter LobeHub — this enables the pre-commit check agent to diagnose CI failures directly without leaving the agent session.

The official GitHub MCP server exposes **70+ tools** organized into 9 toolsets. GitHub The critical confirmation: **`get_job_logs`** accepts `job_id`, `run_id`, and `failed_only` parameters. When `failed_only=true` with a `run_id`, it retrieves logs for all failed jobs in that workflow run. GitHub GitHub's own blog provides the usage example: "Prompt: 'Why did the release.yml job fail last night?' → The agent pulls logs, analyzes the stack trace, and suggests fixes." DevStackTips

Additional capabilities relevant to the workflow: `list_dependabot_alerts` for dependency vulnerability surfacing, DevStackTips `create_pull_request` with structured descriptions, full issue CRUD as a complement to Linear, and CI status querying via `get_workflow_run` and `list_workflow_runs`. Skypage The server supports **read-only mode** (`--read-only` flag) for safety-critical environments. LobeHub

Authentication uses OAuth (recommended for the remote server) or Personal Access Tokens. GitHub Rate limits are GitHub's standard **5,000 requests/hour** authenticated. The server is transitioning from legacy per-endpoint tools to consolidated tools behind a feature flag — currently the remote server returns legacy tool names.

Anti-pattern: Loading all 70+ tools at session start. With each tool definition costing 550–1,400 tokens, the full GitHub MCP server consumes significant context before any work begins. Use `--toolsets` filtering or the dynamic toolset discovery feature (meta-tools: `listAvailableToolsets`, `enableToolset`, `getToolsetTools`) to load only what the current session needs. LobeHub

PostgreSQL MCP should use PG-MCP for pgvector support with strict read-only enforcement

Top recommendation: Use **PG-MCP** (`stuzero/pg-mcp-server`) for development and staging environments — it is the only MCP server with explicit pgvector awareness through YAML-based extension knowledge, plus multi-database support via secure connection IDs. Enforce read-only transactions at the server level. **Never connect an MCP server to production databases** under any circumstances.

The PostgreSQL MCP ecosystem ranges from Anthropic's reference implementation (now archived, GitHub basic schema discovery and read-only queries only) Warp to the writenotenow/postgres-mcp with 232 specialized tools. For the described stack requiring pgvector, the landscape narrows significantly:

- **PG-MCP** — explicit pgvector and PostGIS extension context via YAML knowledge bases, multi-database connections, query explanation via `pg_explain`, read-only by default
- **writenotenow/postgres-mcp** — supports pgvector among 8+ extensions, includes migration tracking and connection pooling, configurable read/write access
- **pgEdge** — production-grade multi-database support with `pg_stat_statements` integration, but no specific pgvector awareness

For **extension verification** (uuid-ossp, pgvector, pg_trgm), PG-MCP and writenotenow/postgres-mcp both support querying `pg_available_extensions` and `pg_extension`. Migration state inspection is only available in writenotenow/postgres-mcp. Connection pool status is available in writenotenow/postgres-mcp and pgEdge.

The **security model** must enforce strict environment separation. Practitioner consensus is unambiguous: "I tend to avoid connecting via MCP a production database under any circumstance because we are directly executing queries generated by Gen AI" (punits.dev). The Cerbos blog warns: "All it takes is one errant command for an AI agent to drop your production database." Recommended pattern: separate database users with minimal privileges per environment, read-only transactions enforced at server level, network isolation for MCP servers, and human-in-the-loop for any write operations.

Anti-pattern: Using a single MCP connection string across environments. PG-MCP's connection ID system and pgEdge's multi-database support provide the mechanism, but the configuration

must enforce that production credentials are never available in the agent's environment. Use environment-scoped tool whitelisting: full access for development, read-only for staging, no access for production.

MCP invocation should use dynamic loading with code-execution for data-heavy operations

Top recommendation: Default to reactive/dynamic tool loading rather than front-loading all MCP tools at session start. Use Claude Code's built-in Tool Search feature (limits tool definitions to 10% of context), enable toolset filtering on all MCP servers, and use the code-execution pattern for data-heavy operations to achieve **98%+ token savings** on results.

The core problem is quantified: each MCP tool definition costs **550–1,400 tokens** on every API call regardless of use. One practitioner reported 3 MCP servers consuming **143,000 of 200,000 tokens (72%)** before any conversation began. A Scalekit benchmark showed MCP costing **4–32× more tokens than CLI** for identical operations.

Anthropic's recommended approach is the code-execution pattern (documented November 2025): instead of direct tool calls, the agent writes code that calls MCP tools, filters results in the execution environment, and returns only relevant data to the model context. Example: 150,000 tokens of raw database query results become 2,000 tokens after code-based filtering — a **98.7% reduction**. This pattern also provides privacy benefits, as intermediate results never enter the model's context.

For this workflow specifically, the **session-type-based loading pattern** is recommended:

- **Planning sessions**: Load Linear MCP + Figma MCP, disable GitHub and PostgreSQL
- **Implementation sessions**: Load GitHub MCP (repos + actions toolsets) + PostgreSQL MCP, disable Linear and Figma
- **Review sessions**: Load GitHub MCP (pull_requests + code_security) + Linear MCP, disable Figma and PostgreSQL

Claude Code's feature loading hierarchy provides the mechanism: CLAUDE.md and MCP servers load at session start (expensive), skills load only descriptions initially (cheap, ~100 tokens each), subagents run in isolated context (no main-context cost), and hooks execute externally at zero context cost. The Context Mode MCP Server (`mksglu/context-mode`) virtualizes tool output to reduce consumption by 98%, extending session time from ~30 minutes to ~3 hours.

Anti-pattern: Front-loading all 4 MCP servers in every session. With Linear (25 tools), Figma (12 tools), GitHub (70+ tools), and PostgreSQL (varies by server) loaded simultaneously, tool definitions alone could consume 50,000+ tokens before any work begins, degrading model performance on the actual task.

Consolidated recommendations in priority order

The recommendations below are ranked by effort-to-impact ratio, with the highest-leverage, lowest-effort items first.

Tier 1 — Configure immediately (high impact, low effort):

1. **Continuation Prompt Skill** — implement the Document and Clear pattern with the state-snapshot-plus-pointer handoff structure. This compounds across every session and prevents the most expensive failure mode (context loss leading to rework). Use `.claude/handoffs/` with timestamps.
2. **AGENTS.md with progressive disclosure** — create the root file under 200 lines covering tech stack, build/test commands, and project purpose. Point to `docs/context/` for architectural decisions, schema documentation, and API specifications. Symlink to CLAUDE.md.
3. **MCP toolset filtering** — configure session-type-based loading to prevent context pollution. Enable `--toolsets` on GitHub MCP, `--tool-filter` on PostgreSQL MCP, and use `mcpick` or similar for session-specific server toggling.

Tier 2 — Configure within the first sprint (high impact, moderate effort):

4. **Security Diagnosis Agent** (hallucinated packages) — integrate Socket.dev MCP and implement the package verification checklist. This addresses the highest-severity supply chain risk with immediate ROI.
5. **Pre-commit Check Agent** — configure Claude Code custom mode with Edit/Write tools excluded, structured error output format, and PostToolUse hooks for quality script execution.
6. **Security Review Agent** — set up the hybrid SAST+LLM pattern with Bandit/pip-audit/Semgrep output as input. Map OWASP Top 10 categories into the system prompt.

Tier 3 — Configure iteratively (moderate impact, higher effort):

7. **PRD Writing and PRD-to-Issues Skills** — implement Socratic elicitation with few-shot ticket generation using tool calling and Pydantic validation.
8. **Architecture Review Agent** — requires building the context pipeline (ERD extraction, dependency mapping, ADR collection) before the agent is useful.
9. **C.L.E.A.R. Code Review Skill** — formalize the review checklist with AI-specific dimensions and integrate automated pre-checks.
10. **TDD Review Agent** and **Performance Diagnosis Agent** — lowest priority because tooling maturity is lowest; build incrementally as the workflow matures.

Open questions where industry consensus does not exist

No controlled specialist-vs-generalist comparison exists for production software engineering. The AgentCoder and MapCoder ablation studies demonstrate specialist benefits within their specific architectures, but no published study directly compares the same task solved by a single agent with comprehensive context versus multiple specialist agents with focused context in a controlled production setting. Anthropic's guidance to "start simple" and the Agentless approach's competitive performance suggest the answer may be task-dependent.

The optimal system prompt length is empirically bounded but not precisely defined. The 150–200 instruction limit from HumanLayer's analysis cites a single preprint (arXiv 2507.11538) not yet widely replicated. The 3,000-token reasoning degradation threshold (Levy et al., ACL 2024) applies to reasoning tasks with padding, not specifically to system prompts. Practitioners converge on "under 300 lines" but the exact sweet spot varies by task complexity.

CLAUDE.md system-reminder wrapping behavior is documented by proxy analysis but not officially confirmed by Anthropic. HumanLayer's finding that Claude Code injects a "may or may not be relevant" wrapper around CLAUDE.md content is consequential for instruction file design but has not appeared in Anthropic's public documentation.

TDD review at the planning gate lacks production tooling. No production-grade, widely-adopted tool specifically validates test plans before implementation. The Meridian RFC and requirements verification megaprompts represent early approaches, but the gap between academic acceptance criteria validation and production TDD review remains open.

MCP prefetch patterns have no formal specification. While Anthropic documents the code-execution pattern and Claude Code's Tool Search feature, there is no published standard for session-type-based MCP loading profiles or a formal "MCP prefetch" specification analogous to continuation prompts for file context. Current approaches (mcpick, `--toolsets`, environment-scoped configs) are ad-hoc.

Database MCP security for multi-environment workflows lacks a mature access control model. While all practitioners agree on never connecting to production, the mechanisms for enforcing this (environment-scoped tool whitelisting, connection ID isolation, network segmentation) are implemented differently across servers with no standard approach. The Cerbos and MintMCP approaches to tool-level permission scoping are promising but not yet widely adopted.